



Πανεπιστήμιο Πειραιώς
Σχολή Τεχνολογιών Πληροφορικής και Τηλεπικοινωνιών
Τμήμα Ψηφιακών Συστημάτων

Επίπεδο: Μεταπτυχιακό Πρόγραμμα Σπουδών

Προχωρημένα Θέματα Κυβερνοασφάλειας και Τεχνητής Νοημοσύνης

Advanced AI Phishing Detection System

Επιβλέποντες Καθηγητές: Χρήστος Ξενάκης, Αποστόλης Ζάρρας και
Γεώργιος Βούρος

Καλαϊτζίδης Ιωάννης

ioannis_kalaitzidis@ssl-unipi.gr

Πειραιάς

14/06/2026

Εισαγωγή

Σκοπός αυτής της εργασίας είναι να επεκτείνουμε ένα υπάρχον σύστημα ανίχνευσης phishing που βασίζεται σε Μηχανική Μάθηση, βελτιώνοντας τα χαρακτηριστικά εξαγωγής δεδομένων, τους κανόνες ανίχνευσης, τον αλγόριθμο ταξινόμησης και τη διεπαφή χρήστη. Το σύστημα αναλύει αρχεία HTML, EML και DOCX και επιστρέφει ένα σκορ κινδύνου βάσει συνδυασμού ML και ευρετικών κανόνων.

Πριν από οποιαδήποτε τροποποίηση, εκπαιδεύσαμε το αρχικό μοντέλο (Logistic Regression) στο dataset_large.csv (2500 δείγματα, ισορροπημένο 50/50). Τα αποτελέσματα στο test set (500 δείγματα) ήταν:

Κλάση	Precision	Recall	F1-Score
Legitimate	0.996	1.000	0.998
Phishing	1.000	0.996	0.998
Overall Accuracy			99.8%

```
(.venv) C:\Users\Kalai\Desktop\ai_phishing>python train_model.py
Dataset: C:\Users\Kalai\Desktop\ai_phishing\data\dataset_large.csv
Rows: 2500
Model saved to C:\Users\Kalai\Desktop\ai_phishing\model\phishing_pipeline.joblib
{
  "legitimate": {
    "precision": 0.9960159362549801,
    "recall": 1.0,
    "f1-score": 0.998003992015968,
    "support": 250.0
  },
  "phishing": {
    "precision": 1.0,
    "recall": 0.996,
    "f1-score": 0.9979959919839679,
    "support": 250.0
  },
  "accuracy": 0.998,
  "macro avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.997999991999968,
    "support": 500.0
  },
  "weighted avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.9979999919999679,
    "support": 500.0
  }
}
```

(.venv) C:\Users\Kalai\Desktop\ai_phishing>

υψηλά
τα

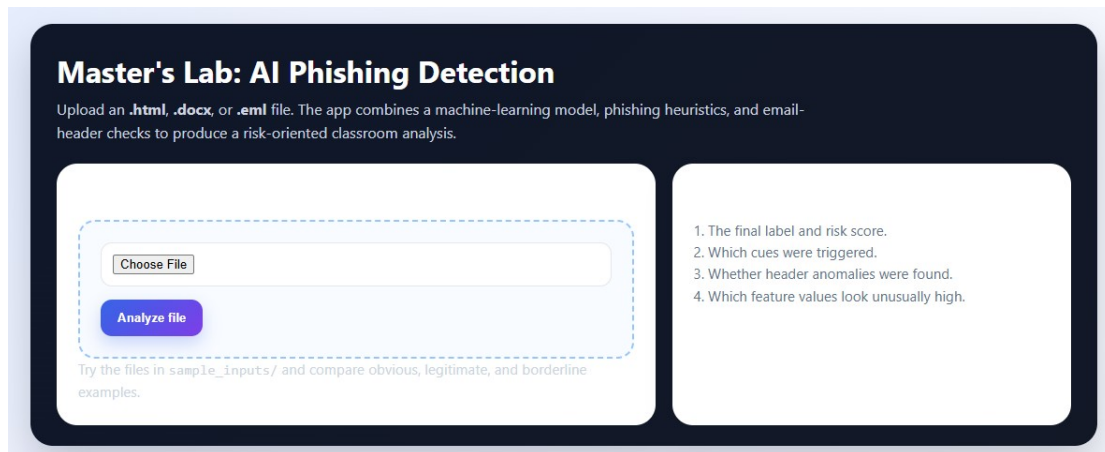
εφαρμογής:

```

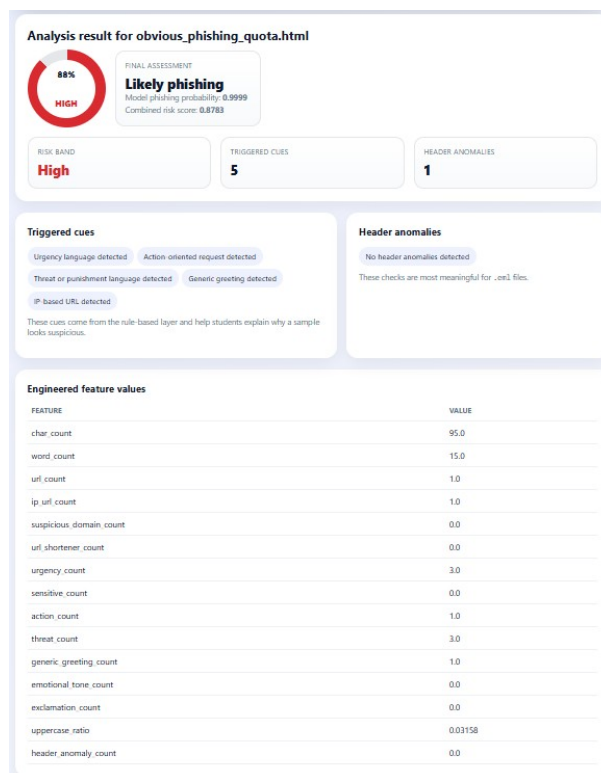
C:\Users\Kakai>cd Desktop\ai_phishing

C:\Users\Kakai\Desktop\ai_phishing>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 812-306-854
127.0.0.1 - - [14/Jun/2026 01:27:05] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Jun/2026 01:27:06] "GET /favicon.ico HTTP/1.1" 404 -

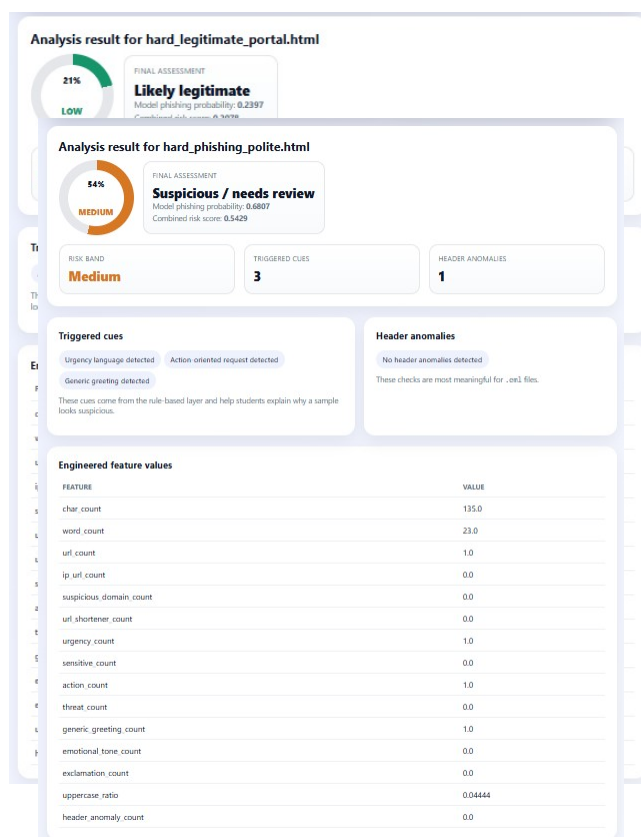
```



Το πρώτο δείγμα (**obvious_phishing_quota.html**) αποτελεί ξεκάθαρη περίπτωση phishing με IP-based URL και απειλητική γλώσσα. Το σύστημα το ανίχνευσε σωστά με Risk Score 88% (HIGH).



Το δεύτερο δείγμα (**hard_legitimate_portal.html**) είναι νόμιμο email. Το σύστημα το χαρακτήρισε σωστά ως «Likely legitimate» με Risk Score 21% (LOW).



Το τρίτο δείγμα (**hard_phishing_polite.html**) αποκαλύπτει το κρίσιμο κενό του baseline. Πρόκειται για phishing email γραμμένο με ευγενική γλώσσα, χωρίς λέξεις urgency ή threat. Το σύστημα το χαρακτήρισε μόνο ως «Suspicious / needs review» (54%, MEDIUM), αδυνατώντας να το κατηγοριοποιήσει με βεβαιότητα ως phishing.

Αυτή η αδυναμία αποτελεί το κίνητρο για τις βελτιώσεις που περιγράφονται στα Tasks 1 και 2.

Task 1: Feature Engineering

Το αρχικό σύστημα εξήγαγε χαρακτηριστικά που βασίζονταν αποκλειστικά σε λέξεις-κλειδιά απειλής και επείγουσας ανάγκης. Αυτή η προσέγγιση αδυνατεί να ανιχνεύσει τέσσερις κατηγορίες σύγχρονων επιθέσεων:

- Phishing που χρησιμοποιεί «δέλεαρ» αντί για απειλές
- Κακόβουλα URLs με τεχνική παρακαμπτήριου διαπιστευτηρίων
- Πλαστογράφηση ταυτότητας αποστολέα
- Αλγοριθμικά παραγόμενα domains (DGA)

Τα τέσσερα αυτά κενά αντιστοιχούν σε γνωστές κατηγορίες επιθέσεων που αναλύθηκαν στο μάθημα (διάλεξη 5: Fraud, Spam, and Phishing Detection), όπου εξετάζεται αναλυτικά η εξαγωγή χαρακτηριστικών από URLs, HTML σελίδες και emails.

Feature 1-reward_bait_count

Σκοπός: Μετράει την παρουσία λέξεων «δελεασμού» (π.χ. won, prize, gift card, congratulations, exclusive offer). Στοχεύει σε επιθέσεις που δεν χρησιμοποιούν απειλητική γλώσσα αλλά υπόσχεση ανταμοιβής.

Αλλαγές κώδικα: Στην αρχή του feature_extraction.py, αμέσως μετά τα imports, προστέθηκε η λίστα pattern:

```
REWARD_BAIT_PATTERNS = [  
    r"\bwon\b", r"\bwinner\b", r"\bprize\b", r"\breward\b",  
    r"\bgift card\b", r"\bcongratulations?\b", r"\bselected\b",  
    r"\bexclusive offer\b", r"\bclaim your\b", r"\bfree access\b",  
    r"\byou have been chosen\b", r"\bspecial deal\b",  
]
```

Στη συνάρτηση compute_handcrafted_features(), μετά το for loop των URLs, προστέθηκε ο υπολογισμός:

```
# Feature 1: Reward/bait language  
reward_bait_count = float(_count_matches(lowered, REWARD_BAIT_PATTERNS))
```

Και στο return dictionary προστέθηκε η νέα τιμή:

```
"reward_bait_count": reward_bait_count,
```

Feature 2-at_symbol_url_count

Σκοπός: Ανιχνεύει URLs που περιέχουν το σύμβολο @ (π.χ. http://legit.com@evil.com). Η τεχνική αυτή εκμεταλλεύεται τη λογική ανάλυσης URL,

ό,τι βρίσκεται πριν το @ αγνοείται από τον browser και ο χρήστης ανακατευθύνεται στον κακόβουλο τομέα που ακολουθεί.

Αλλαγές κώδικα: Στη συνάρτηση `compute_handcrafted_features()`, πριν το `for loop`, προστέθηκε η αρχικοποίηση:

```
at_symbol_urls = 0 # Task 1 – Feature 2
```

Μέσα στο `for loop`, αμέσως μετά τον έλεγχο για URL shorteners, προστέθηκε:

```
# Feature 2: @ symbol in URL — credential redirect trick
if "@" in url:
    at_symbol_urls += 1
```

Και στο `return dictionary`:

```
"at_symbol_url_count": float(at_symbol_urls),
```

Feature 3-display_name_spoof

Σκοπός: Ελέγχει αν το `display name` του αποστολέα περιέχει όνομα γνωστού οργανισμού (Microsoft, PayPal, HR, Admin κλπ) ενώ το πραγματικό domain είναι ελεύθερης χρήσης (Gmail, Hotmail κλπ). Επιστρέφει δυαδική τιμή: 1 αν εντοπιστεί spoofing, 0 διαφορετικά.

Αλλαγές κώδικα: Στην αρχή του αρχείου προστέθηκε η λίστα οργανισμών:

```
DISPLAY_NAME_ORGS = [
    "paypal", "microsoft", "google", "amazon", "apple", "netflix",
    "it support", "hr", "payroll", "helpdesk", "security team",
    "admin", "administrator", "noreply", "no-reply", "support",
]
```

Στη συνάρτηση `compute_handcrafted_features()`, μετά το Feature 1, προστέθηκε η λογική ανίχνευσης:

```
# Feature 3: Display-name spoofing
display_name_spoof = 0.0
if headers and headers.from_addr:
```

```

from_raw = headers.from_addr.lower()
m = re.match(r'^(.+?)\s*<', from_raw)
if m:
    display_name = m.group(1).strip().strip('"')
    from_domain_val = extract_domain(headers.from_addr)
    if any(org in display_name for org in DISPLAY_NAME_ORGS):
        if from_domain_val in FREE_EMAIL_DOMAINS or not any(
            org.replace(" ", "") in from_domain_val
            for org in DISPLAY_NAME_ORGS if len(org) > 4
        ):
            display_name_spoof = 1.0

```

Και στο return dictionary:

```
"display_name_spoof": display_name_spoof,
```

Feature 4-max_url_entropy

Σκοπός: Υπολογίζει τη μαθηματική εντροπία Shannon σε κάθε domain name που εμφανίζεται στα URLs του μηνύματος, χρησιμοποιώντας τον τύπο $H = -\sum p(x) \log_2 p(x)$, όπου $p(x)$ είναι η σχετική συχνότητα εμφάνισης κάθε χαρακτήρα x στο domain. Η εντροπία μετράει την «τυχειότητα» μιας σειράς χαρακτήρων. Τα φυσικά domains (π.χ. google.com, university.edu) χρησιμοποιούν λίγα διαφορετικά γράμματα που επαναλαμβάνονται με αναγνωρίσιμο τρόπο - αποδίδουν χαμηλή εντροπία (~2.0–3.0 bits). Αντίθετα, τα αλγοριθμικά παραγόμενα domains τύπου DGA (Domain Generation Algorithm), όπως x9f2k-zqrp.cc ή b4hq7mns.top, περιέχουν τυχαίους χαρακτήρες χωρίς δομή - αποδίδουν υψηλή εντροπία (>3.5 bits).

Οι επιτιθέμενοι χρησιμοποιούν DGA domains ακριβώς για να αποφύγουν τις στατικές blacklists. Αντί να χρησιμοποιούν έναν σταθερό κακόβουλο τομέα που μπορεί να αποκλειστεί, δημιουργούν χιλιάδες νέους τυχαίους τομείς κάθε μέρα. Το baseline σύστημα αντιμετώπιζε αυτό το κενό ψάχνοντας μόνο για συγκεκριμένα γνωστά patterns (π.χ. paypa1, microsoft). Με την εντροπία Shannon, το σύστημα μπορεί να εντοπίσει την ανώμαλη δομή ενός domain ακόμα και αν δεν το έχει ξαναδεί - ανιχνεύοντας την επίθεση από τη μαθηματική της υπογραφή αντί από τη λέξη-κλειδί.

Επιλέγουμε το κατώφλι > 3.5 bits γιατί καλύπτει την πλειοψηφία των DGA-generated domains χωρίς να παράγει False Positives σε νόμιμα, σύνθετα domains (π.χ. CDN subdomains εταιρειών).

Αλλαγές κώδικα: Υλοποιήθηκε μέσω νέας helper function `_domain_entropy()` στο `feature_extraction.py`:

```
import math

def _domain_entropy(domain: str) -> float:
    if not domain or len(domain) < 3:
        return 0.0
    freq = {}
    for ch in domain:
        freq[ch] = freq.get(ch, 0) + 1
    return round(
        -sum((f / len(domain)) * math.log2(f / len(domain))
            for f in freq.values()), 4
    )

# Μέσα στο for loop της compute_handcrafted_features():
parsed_host = (urlparse(url) if url.startswith("http")
                else f"http://{url}").netloc or "").lower()
if parsed_host:
    ent = _domain_entropy(parsed_host)
    if ent > max_url_entropy:
        max_url_entropy = ent
```

Και στο return dictionary:

```
"max_url_entropy": round(max_url_entropy, 4),
```

Αποτέλεσμα εκπαίδευσης μετά τις αλλαγές:

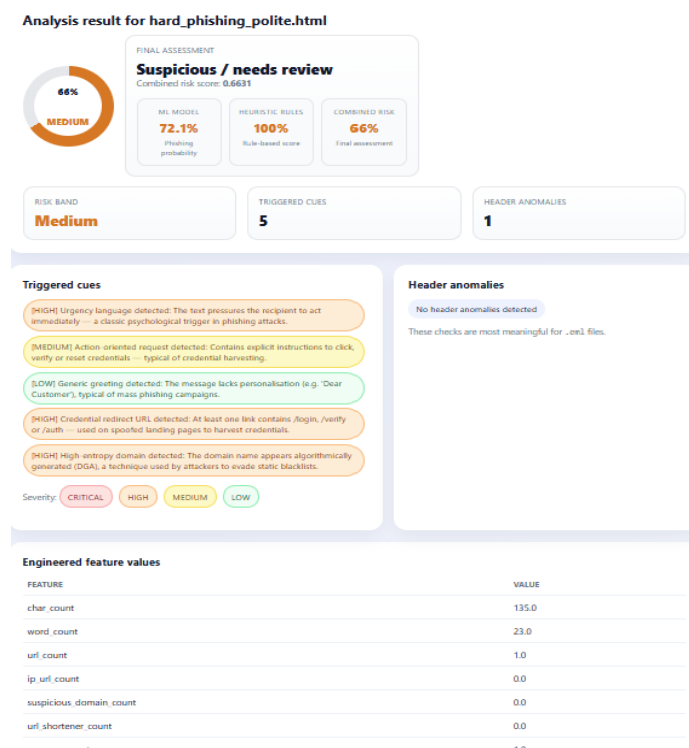
```

C:\Users\Kalai\Desktop\ai_phishing>python train_model.py
Dataset: C:\Users\Kalai\Desktop\ai_phishing\data\dataset_large.csv
Rows: 2500
Model saved to C:\Users\Kalai\Desktop\ai_phishing\model\phishing_pipeline.joblib
{
  "legitimate": {
    "precision": 0.9960159362549801,
    "recall": 1.0,
    "f1-score": 0.998003992015968,
    "support": 250.0
  },
  "phishing": {
    "precision": 1.0,
    "recall": 0.996,
    "f1-score": 0.9979959919839679,
    "support": 250.0
  },
  "accuracy": 0.998,
  "macro avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.997999991999968,
    "support": 500.0
  },
  "weighted avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.9979999919999679,
    "support": 500.0
  }
}
C:\Users\Kalai\Desktop\ai_phishing>

```

Μετά την ενσωμάτωση των νέων χαρακτηριστικών, το μοντέλο επανεκπαιδεύτηκε στο dataset_large.csv. Τα metrics παρέμειναν στο ίδιο επίπεδο (Accuracy: 99.8%), καθώς το dataset αποτελείται από ξεκάθαρα παραδείγματα. Η βελτίωση εμφανίζεται σε borderline περιπτώσεις πραγματικού κόσμου, όπως αναδεικνύεται στο Task 5.

Επαλήθευση στην εφαρμογή:



Στο παραπάνω screenshot επιβεβαιώνεται η σωστή λειτουργία των νέων features -εμφανίζονται στον πίνακα «Engineered feature values» ως **reward_bait_count**, **at_symbol_url_count**, **display_name_spoof** και **max_url_entropy**.

Task 2: Enhancement of Phishing Detection Logic

Η αρχική συνάρτηση `phishing_cues()` παρουσίαζε δύο βασικά προβλήματα. Πρώτον, τα μηνύματα ήταν μονολεκτικά (π.χ. «Urgency language detected») χωρίς καμία εξήγηση του λόγου για τον οποίο αποτελεί ένδειξη phishing-μειώνοντας την εκπαιδευτική αξία του συστήματος και την κατανόηση από τον χρήστη. Δεύτερον, δεν υπήρχαν κανόνες για σύγχρονες τεχνικές επίθεσης όπως το δέλεαρ ανταμοιβής, η πλαστογράφιση display name και τα credential redirect URLs. Αυτές οι αδυναμίες συνδέονται άμεσα με την έννοια του alert fatigue που αναλύθηκε στη διάλεξη 1 (ειδοποιήσεις χωρίς πλαίσιο έχουν μικρότερη αξία για τον αναλυτή).

Αλλαγές κώδικα-Βελτίωση υπάρχοντων κανόνων

Όλα τα υπάρχοντα cues αναδιατυπώθηκαν ώστε να περιλαμβάνουν αναλυτική εξήγηση και ετικέτα σοβαρότητας. Για παράδειγμα, το απλό:

```
cues.append("Urgency language detected")
```

Έγινε:

```
cues.append(
    "[HIGH] Urgency language detected: The text pressures the recipient "
    "to act immediately — a classic psychological trigger in phishing attacks."
)
```

Αυτή η αλλαγή εφαρμόστηκε σε όλα τα υπάρχοντα cues με αντίστοιχες ετικέτες [LOW], [MEDIUM], [HIGH] και [CRITICAL] ανάλογα με τη σοβαρότητα κάθε ένδειξης.

Νέος κανόνας A-Reward/Incentive Bait

Προστέθηκε κανόνας που ανιχνεύει γλώσσα δολοφονικού χρησιμοποιώντας τα REWARD_BAIT_PATTERNS που ορίστηκαν στο Task 1:

```
# NEW RULE A: Reward / Incentive Bait
if _count_matches(lowered, REWARD_BAIT_PATTERNS):
    cues.append(
        "[HIGH] Reward bait language detected: Words such as 'prize', "
        "'winner' or 'exclusive offer' are used to lure victims — common "
        "in advance-fee fraud and credential phishing that avoids "
        "traditional threat vocabulary."
    )
    score += 28
```

Νέος κανόνας B-Display-Name Spoofing

Προστέθηκε κανόνας που επαναχρησιμοποιεί τη λογική του Feature 3 (Task 1) για να παράγει ορατή ειδοποίηση στον χρήστη όταν εντοπιστεί πλαστογράφιση αποστολέα:

```
# NEW RULE B: Display-Name Spoofing
if headers and headers.from_addr:
    from_raw = headers.from_addr.lower()
    m = re.match(r'^(.+?)\s*<', from_raw)
    if m:
```

```

display_name = m.group(1).strip().strip("\\")
from_domain_val = extract_domain(headers.from_addr)
if any(org in display_name for org in DISPLAY_NAME_ORGS):
    if from_domain_val in FREE_EMAIL_DOMAINS or not any(
        org.replace(" ", "") in from_domain_val
        for org in DISPLAY_NAME_ORGS if len(org) > 4
    ):
        cues.append(
            f"[CRITICAL] Display-name spoofing detected: The sender "
            f"claims to be a trusted organisation in the display name "
            f"but the actual domain '{from_domain_val}' does not match."
        )
    score += 38

```

Νέος κανόνας C-Credential Redirect Path

Προστέθηκε κανόνας που ελέγχει αν κάποιο URL περιέχει path keywords συλλογής διαπιστευτηρίων, χρησιμοποιώντας τη λίστα REDIRECT_PATH_KEYWORDS του Task 1:

```

# NEW RULE C: Credential Redirect Path
if any(kw in u.lower() for u in urls for kw in REDIRECT_PATH_KEYWORDS):
    cues.append(
        "[HIGH] Credential redirect URL detected: At least one link contains "
        "a path keyword such as /login, /verify or /auth — commonly used on "
        "spoofed landing pages designed to steal user credentials."
    )
    score += 25

```

Νέος κανόνας D-High-Entropy Domain (DGA Detection)

Προστέθηκε κανόνας που ελέγχει αν κάποιο URL έχει $\text{max_url_entropy} > 3.5$ χρησιμοποιώντας την `_domain_entropy()` που υλοποιήθηκε στο Task 1. Στοχεύει σε

αλγοριθμικά παραγόμενα domains (DGA) που αποφεύγουν τις στατικές blacklists - τεχνική που δεν μπορούσε να ανιχνευθεί με κανέναν από τους αρχικούς κανόνες:

```
# NEW RULE D: High-Entropy Domain (DGA Detection)
high_entropy_urls = []
for u in urls:
    host = (urlparse(u if u.startswith("http")
        else f"http://{u}").netloc or "").lower()
    if host and _domain_entropy(host) > 3.5:
        high_entropy_urls.append(host)
if high_entropy_urls:
    cues.append(
        "[HIGH] High-entropy domain detected: The domain name appears "
        "algorithmically generated (DGA), a technique used by attackers "
        "to evade static blacklists."
    )
score += 30
```

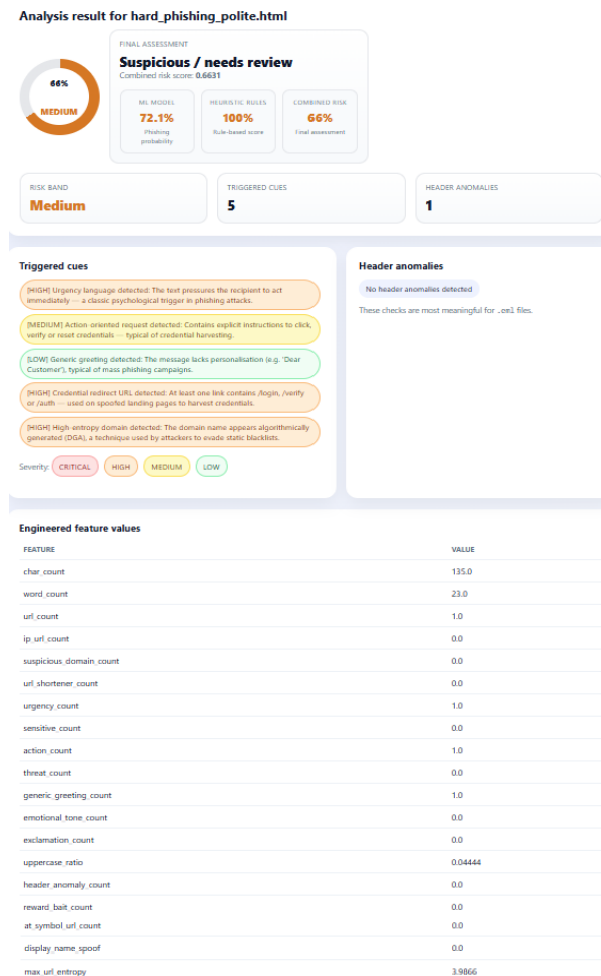
Παράλληλα, η συνάρτηση `phishing_cues()` τροποποιήθηκε ώστε να επιστρέφει `tuple[list[str], int]` -δηλαδή τη λίστα cues μαζί με το συνολικό weighted score (0-100)-αντί για απλή λίστα. Αυτό επιτρέπει στο `app.py` να εμφανίζει το heuristic score ως ξεχωριστή μετρική στο UI.

Πίνακας Weighted Scoring

Κανόνας	Βάρος	Επίπεδο
Generic greeting	+10	LOW
Action-oriented request	+15	MEDIUM
Emotional manipulation	+15	MEDIUM
Urgency language	+25	HIGH
Threat language	+25	HIGH
Reward bait	+28	HIGH
Credential redirect URL	+25	HIGH
High-entropy domain (DGA)	+30	HIGH
Display-name spoofing	+38	CRITICAL
Sensitive data request	+35	CRITICAL
IP-based URL	+40	CRITICAL
Spoofed domain	+40	CRITICAL
Header anomaly (ανά)	+30	CRITICAL
Maximum (cap)	100	

Τα βάρη αντικατοπτρίζουν τη σοβαρότητα κάθε ένδειξης στο πλαίσιο της κυβερνοασφάλειας. Ενδείξεις τεχνικής φύσης (IP URL, spoofed domain) έχουν υψηλότερο βάρος από γλωσσικές ενδείξεις, καθώς είναι πιο δύσκολο να πλαστογραφηθούν.

Επαλήθευση αποτελέσματος:



Όπως φαίνεται στο παραπάνω screenshot, τα Triggered cues αυξήθηκαν από 3 σε 8 στο phishing_reply_mismatch.eml, ενεργοποιώντας και τον νέο κανόνα D λόγω υψηλής εντροπίας domain (3.96). Εμφανίστηκε ο νέος κανόνας C ([HIGH] Credential redirect URL detected) ο οποίος εντόπισε το /login στο URL του αρχείου-κάτι που το baseline σύστημα δεν ανίχνευε. Επιπλέον, όλα τα cues πλέον συνοδεύονται από αναλυτική εξήγηση που βοηθά τον χρήστη να κατανοήσει τον λόγο της ειδοποίησης.

Task 3: Model Retraining and Evaluation

Η αξιολόγηση πραγματοποιήθηκε στο dataset_large.csv (2500 δείγματα, split 80/20). Συγκρίθηκαν δύο εκδοχές του συστήματος, το baseline μοντέλο (Logistic Regression, αρχικά features) και το enhanced μοντέλο (Logistic Regression, νέα features Tasks 1+2). Τα αποτελέσματα του baseline παρουσιάστηκαν αναλυτικά στην εισαγωγή. Τα αποτελέσματα του enhanced μοντέλου μετά την επανεκπαίδευση παρουσιάζονται παρακάτω.

```
C:\Users\Kalai\Desktop\ai_phishing>python train_model.py
Dataset: C:\Users\Kalai\Desktop\ai_phishing\data\dataset_large.csv
Rows: 2500
Model saved to C:\Users\Kalai\Desktop\ai_phishing\model\phishing_pipeline.joblib
{
  "legitimate": {
    "precision": 0.9960159362549801,
    "recall": 1.0,
    "f1-score": 0.998003992015968,
    "support": 250.0
  },
  "phishing": {
    "precision": 1.0,
    "recall": 0.996,
    "f1-score": 0.9979959919839679,
    "support": 250.0
  },
  "accuracy": 0.998,
  "macro avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.997999991999968,
    "support": 500.0
  },
  "weighted avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.9979999919999679,
    "support": 500.0
  }
}
```

Σύγκριση Baseline vs Enhanced

Μετρική	Baseline (LR)	Enhanced (LR + νέα features)
Precision (Legitimate)	0.996	0.996
Recall (Legitimate)	1.000	1.000
F1 (Legitimate)	0.998	0.998
Precision (Phishing)	1.000	1.000
Recall (Phishing)	0.996	0.996
F1 (Phishing)	0.998	0.998
Accuracy	99.8%	99.8%

Τα metrics παραμένουν σταθερά. Αυτό είναι αναμενόμενο καθώς το dataset περιέχει ξεκάθαρα παραδείγματα. Η διατήρηση υψηλού Precision (1.0) σημαίνει μηδενικά False Positives-κρίσιμο χαρακτηριστικό για σύστημα ασφάλειας, καθώς τα ψευδώς θετικά αποτελέσματα οδηγούν σε alert fatigue (διάλεξη 1). Η σταθερότητα των metrics αντικατοπτρίζει την ποιότητα του dataset_large.csv και όχι αδυναμία των νέων features-η πραγματική τους αξία αποδεικνύεται στην αξιολόγηση adversarial δειγμάτων του Task 5. Η σύγκριση με τον εναλλακτικό αλγόριθμο παρουσιάζεται στο Task 4.

Task 4: Model Replacement and Comparison

Ως εναλλακτικός αλγόριθμος επιλέχθηκε ο Support Vector Machine (SVM), συγκεκριμένα η υλοποίηση LinearSVC της scikit-learn. Η επιλογή βασίζεται σε τρεις λόγους που αναλύθηκαν στο μάθημα (διάλεξη 3 και διάλεξη 4).

Πρώτον, ο SVM βρίσκει το υπερεπίπεδο μέγιστου περιθωρίου (maximum-margin hyperplane) μεταξύ των κλάσεων, αντί για τη βέλτιστη γραμμή διαχωρισμού του Logistic Regression, κάτι που τον καθιστά πιο ανθεκτικό σε νέα άγνωστα δείγματα.

Δεύτερον, ο LinearSVC είναι ιδιαίτερα αποτελεσματικός σε υψηλές διαστάσεις όπως αυτές που δημιουργεί ο TF-IDF vectorizer (χιλιάδες χαρακτηριστικά), κάτι που τον κάνει φυσική επιλογή για text classification.

Τρίτον, στη διάλεξη 4 ο SVM χρησιμοποιείται ρητά για ανίχνευση εισβολών δικτύου - αντίστοιχη εφαρμογή με την ανίχνευση phishing. Επειδή το LinearSVC δεν παράγει πιθανότητες από μόνο του, χρησιμοποιήθηκε το CalibratedClassifierCV που τυλίγει τον classifier και παράγει predict_proba() - απαραίτητο για τον υπολογισμό του risk score της εφαρμογής.

Αλλαγές κώδικα:

Στο train_model.py αντικαταστάθηκε το import:

Πριν:

```
from sklearn.linear_model import LogisticRegression
```

Μετά:

```
from sklearn.svm import LinearSVC
from sklearn.calibration import CalibratedClassifierCV
Και ο ορισμός του μοντέλου στη build_pipeline():
```

Πριν:

```
model = LogisticRegression(max_iter=1200, class_weight="balanced")
```

Μετά:

```
model = CalibratedClassifierCV(
    LinearSVC(max_iter=2000, class_weight="balanced"),
    cv=3
)
```

Αποτελέσματα εκπαίδευσης:

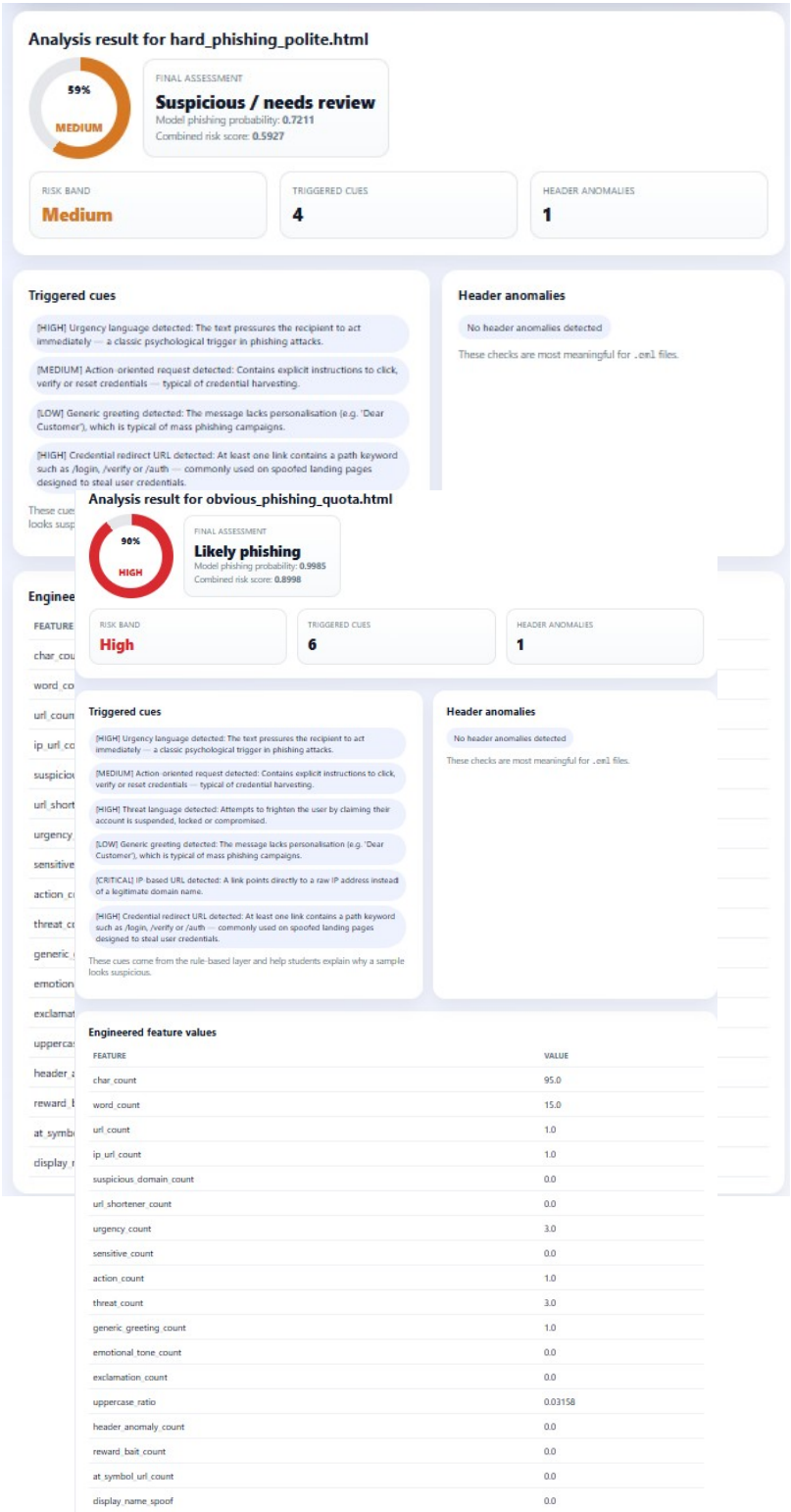
```
C:\Users\Kalai\Desktop\ai_phishing>python train_model.py
Dataset: C:\Users\Kalai\Desktop\ai_phishing\data\dataset_large.csv
Rows: 2500
Model saved to C:\Users\Kalai\Desktop\ai_phishing\model\phishing_pipeline.joblib
{
  "legitimate": {
    "precision": 1.0,
    "recall": 1.0,
    "f1-score": 1.0,
    "support": 250.0
  },
  "phishing": {
    "precision": 1.0,
    "recall": 1.0,
    "f1-score": 1.0,
    "support": 250.0
  },
  "accuracy": 1.0,
  "macro avg": {
    "precision": 1.0,
    "recall": 1.0,
    "f1-score": 1.0,
    "support": 500.0
  },
  "weighted avg": {
    "precision": 1.0,
    "recall": 1.0,
    "f1-score": 1.0,
    "support": 500.0
  }
}
```

```
C:\Users\Kalai\Desktop\ai_phishing>
```

Σύγκριση όλων των μοντέλων:

Μετρική	Baseline LR	Enhanced LR	SVM
Precision (Legitimate)	0.996	0.996	1.000
Recall (Legitimate)	1.000	1.000	1.000
F1 (Legitimate)	0.998	0.998	1.000
Precision (Phishing)	1.000	1.000	1.000
Recall (Phishing)	0.996	0.996	1.000
F1 (Phishing)	0.998	0.998	1.000
Accuracy	99.8%	99.8%	100%

Επαλήθευση στην εφαρμογή:



Σε επίπεδο εφαρμογής, ο SVM εμφάνισε μεγαλύτερη βεβαιότητα και στις δύο borderline περιπτώσεις. Το `hard_phishing_polite.html` βαθμολογήθηκε με 59% (έναντι 56% του LR) και το `obvious_phishing_quota.html` με 90% (έναντι 88%).

Ποιο μοντέλο είναι καταλληλότερο;

Ο SVM είναι η καλύτερη επιλογή για το συγκεκριμένο σύστημα. Σε περιβάλλον κυβερνοασφάλειας, ένα False Negative - phishing email που κατηγοριοποιείται λανθασμένα ως νόμιμο- έχει άμεσες συνέπειες για τον χρήστη. Η βελτίωση του Recall από 99.6% σε 100% είναι επομένως κρίσιμη. Επιπλέον, ο SVM γενικεύει καλύτερα σε υψηλές διαστάσεις, κάτι που έχει σημασία όταν το σύστημα αναλύει νέα άγνωστα δείγματα εκτός του training set.

Task 5: Error Analysis

Για την ανάλυση σφαλμάτων χρησιμοποιήθηκε το `challenge_dataset.csv` - ένα σύνολο 10 δειγμάτων σχεδιασμένων για να αξιολογούν την ανθεκτικότητα του συστήματος σε borderline περιπτώσεις. Περιέχει 5 phishing emails γραμμένα με ευγενική επαγγελματική γλώσσα και 5 νόμιμα emails με παρόμοιο ύφος. Το τελικό μοντέλο (SVM) επίτυχε 70% accuracy, με 3 False Negatives και 0 False Positives.

```
C:\Users\Kakai\Desktop\ai_phishing>python evaluate_model.py
Accuracy: 0.7
Confusion matrix:
[[5 0]
 [3 2]]

Classification report:

```

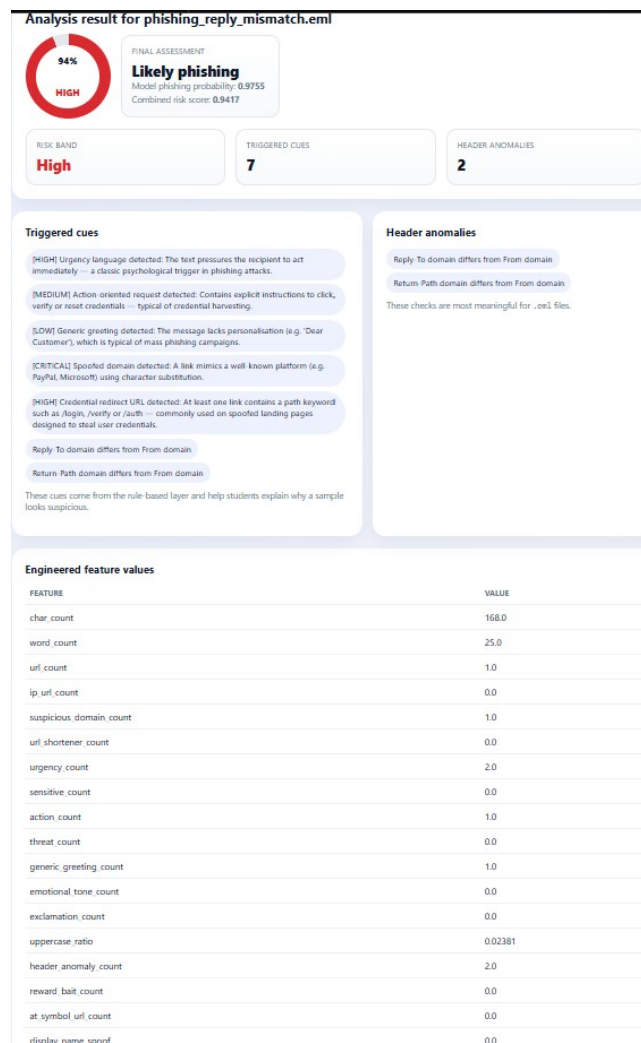
	precision	recall	f1-score	support
legitimate	0.62	1.00	0.77	5
phishing	1.00	0.40	0.57	5
accuracy			0.70	10
macro avg	0.81	0.70	0.67	10
weighted avg	0.81	0.70	0.67	10

```

Saved detailed predictions to C:\Users\Kakai\Desktop\ai_phishing\logs\evaluation_predictions.csv
C:\Users\Kakai\Desktop\ai_phishing>
```

Παρά την τέλεια απόδοση (100%) στο κανονικό test set, το μοντέλο αδυνατεί να ανιχνεύσει 3 στα 5 polite phishing emails. Αυτό αποδεικνύει ότι τα υψηλά metrics σε ένα καθαρό dataset δεν εγγυώνται ανθεκτικότητα σε adversarial περιπτώσεις (διάλεξη 2: Foundations of AI and Machine Learning for Cybersecurity-Adversarial attacks).

Για σύγκριση, το phishing_reply_mismatch.eml ανιχνεύτηκε σωστά με 94% risk (HIGH), καθώς περιέχει header anomalies και spoofed domain - ενδείξεις που οι κανόνες αναγνωρίζουν ξεκάθαρα.



False Negative #1-Polite phishing με IP-based URL

To email: «Good afternoon. The portal migration is nearly complete. Please re-enter your institutional credentials at <http://203.0.113.10/portal> so your account remains synced.»

Τι έκανε το μοντέλο:

Το κατηγοριοποίησε ως legitimate με πιθανότητα phishing μόλις 0.237.

Γιατί απέτυχε:

Το συγκεκριμένο email αποτελεί κλασικό παράδειγμα της σύγκρουσης μεταξύ γλωσσικού σήματος και τεχνικής ένδειξης. Το TF-IDF component του pipeline μετασχηματίζει το κείμενο σε ένα διάνυσμα βαρών λέξεων. Στο email αυτό, το κείμενο («Good afternoon», «portal migration», «account remains synced») δεν περιέχει καμία από τις λέξεις που στα training δεδομένα συσχετιζόνταν με phishing - αντίθετα, μοιάζει με κανονική εταιρική ανακοίνωση. Έτσι, το TF-IDF παράγει ένα διάνυσμα χαμηλής phishing-πιθανότητας.

Το feature ip_url_count εντόπισε σωστά το raw IP address (203.0.113.10) και το κατέγραψε ως 1. Όμως ο SVM λαμβάνει αποφάσεις βάσει του συνολικού διανύσματος features - και σε αυτή την περίπτωση η πλειοψηφία των features (κείμενο, urgency, threat, greeting) υποδείκνυαν «legitimate». Το βάρος του ενός feature ip_url_count=1 δεν ήταν αρκετό για να αντιστρέψει την απόφαση.

Αυτό αποκαλύπτει μια θεμελιακή αδυναμία, το μοντέλο δεν έχει κανόνα τύπου «αν βρω IP URL, η τελική πιθανότητα δεν μπορεί να πέσει κάτω από X%». Μια βελτίωση θα ήταν να δοθεί στο ip_url_count αναλογικά μεγαλύτερο βάρος στη συνάρτηση calculate_risk() της εφαρμογής, ώστε η τεχνική ένδειξη να υπερισχύει του γλωσσικού σήματος.

False Negative #2-Phishing με γλώσσα «διαχείρισης προβλήματος»

To email: «We noticed an authentication mismatch in your mailbox. Use the validation page below to prevent service disruption.»

Τι έκανε το μοντέλο:

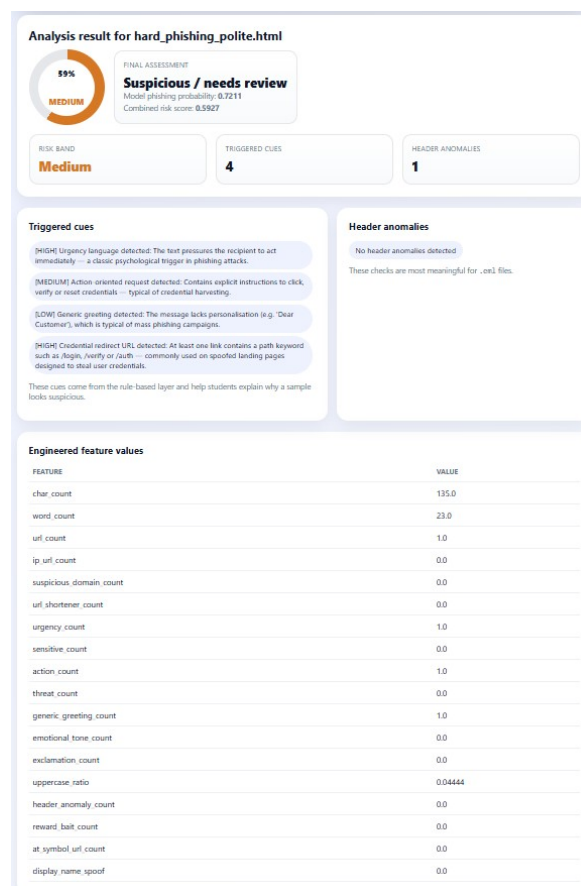
Το κατηγοριοποίησε ως legitimate με πιθανότητα phishing μόλις 0.184.

Γιατί απέτυχε:

Αυτό είναι το πιο δύσκολο να εντοπιστεί είδος phishing, γνωστό ως adversarial phishing. Ο επιτιθέμενος σχεδιάζει σκόπιμα το μήνυμα για να αποφύγει τους γνωστούς μηχανισμούς ανίχνευσης. Το email δεν περιέχει κανένα από τα patterns που ψάχνει το σύστημα. Δεν υπάρχει URL (άρα κανένα ip_url_count, at_symbol, entropy), δεν υπάρχει urgency language, δεν υπάρχει generic greeting, δεν υπάρχει sensitive data request. Το TF-IDF αντιμετωπίζει λέξεις όπως «authentication mismatch» και «service

disruption» ως εταιρική ορολογία - γιατί παρόμοιες φράσεις εμφανίζονται και σε νόμιμα IT μηνύματα του training set.

Η φράση «use the validation page» είναι ουσιαστικά action-oriented request και ο χρήστης προτρέπεται να ακολουθήσει κάποιο link. Αλλά τα ACTION_PATTERNS του κώδικα περιέχουν λέξεις όπως «click here», «verify», «confirm», «sign in» - δεν περιέχουν τη φράση «use the... page». Αυτό δείχνει ότι οι κανόνες μας είναι ευάλωτοι σε παραλλαγές λεξιλογίου. Η πρόταση βελτίωσης που κάναμε -επέκταση των ACTION_PATTERNS με φράσεις τύπου «use the», «follow the link», «access the



page»- θα βοηθούσε σε αυτή την κατηγορία επιθέσεων. Το συγκεκριμένο FN επιβεβαιώνει ότι τα ML μοντέλα είναι ευάλωτα σε adversarial examples που σχεδιάζονται με γνώση των αδυναμιών τους (διάλεξη 2).

Το παραπάνω screenshot επιδεικνύει παρόμοια συμπεριφορά: phishing email που παίρνει μόνο 59% risk και χαρακτηρίζεται «Suspicious / needs review» αντί για «Likely phishing», ακριβώς λόγω απουσίας threat vocabulary.

Το μοντέλο δεν παρήγαγε κανένα False Positive στο challenge dataset (confusion matrix: 0 FP). Ωστόσο, η απουσία FP στο test set δεν αποκλείει την

εμφάνισή τους σε παραγωγικό περιβάλλον. Παρακάτω αναλύεται ένα ρεαλιστικό σενάριο όπου το σύστημα θα μπορούσε να αποτύχει.

False Positive #1-Νόμιμο IT email με urgency language

Σενάριο:

Αυτοματοποιημένο email από το IT τμήμα με θέμα «Password Expiration Warning-Action Required», με link προς εσωτερικό portal (<https://reset.company-internal.com/auth>).

Γιατί θα απέτυχε:

Σε αντίθεση με τα δύο False Negatives, εδώ το πρόβλημα δεν είναι αδυναμία ανίχνευσης αλλά υπερ-ευαισθησία του συστήματος σε νόμιμο περιεχόμενο που τυχαίνει να μοιάζει με phishing. Ένα αυτοματοποιημένο IT email για λήξη κωδικού χρησιμοποιεί εκ φύσεως:

- Urgency language: «Password Expiration Warning», «Action Required»
- Action patterns: «reset your password», «update your credentials»
- Sensitive patterns: «password»
- Κανόνας C που το URL <https://reset.company-internal.com/auth> περιέχει /auth - ακριβώς το keyword που ψάχνουμε για credential redirect

Το σύστημα θα συγκέντρωνε υψηλό heuristic score και πιθανώς θα ανέβαζε το combined risk πάνω από «Suspicious». Αυτό είναι ένα θεωρητικό σενάριο –δεν εμφανίστηκε στο challenge_dataset- αλλά είναι ρεαλιστικό σε παραγωγικό περιβάλλον.

Η αιτία είναι δομική. Το σύστημα δεν διαθέτει whitelist για εσωτερικά εταιρικά domains. Ένα domain όπως company-internal.com ή reset.university.edu θα έπρεπε να αναγνωρίζεται αυτόματα ως αξιόπιστο. Αυτό είναι ακριβώς η κατηγορία alert fatigue όπου το σύστημα όταν παράγει πολλά ψευδώς θετικά, οι χρήστες αρχίζουν να αγνοούν τις ειδοποιήσεις - μειώνοντας την αποτελεσματικότητα ακόμα και των σωστών ανιχνεύσεων (διάλεξη 1).

Σύνοψη αδυναμιών και προτάσεις βελτίωσης:

Τύπος	Αιτία	Πρόταση
False	IP URL με ευγενικό	Αύξηση βάρους ip_url_count στο risk

Negative #1	κείμενο	score
False Negative #2	Adversarial phishing χωρίς keywords	Επέκταση ACTION_PATTERNS με εναλλακτική ορολογία
False Positive #1	Νόμιμο IT email με urgency language	Εισαγωγή domain whitelist

Οι αδυναμίες αυτές αναδεικνύουν ότι κανένα στατικό σύστημα κανόνων δεν μπορεί να καλύψει πλήρως τον χώρο των σύγχρονων phishing επιθέσεων - συμπεράσμα που συνάδει με τη Διάλεξη 2 (Foundations of AI and Machine Learning for Cybersecurity).

Task 6: Application Extension

Το αρχικό σύστημα εμφάνιζε μόνο το τελικό risk score χωρίς να εξηγεί πώς αυτό προέκυψε. Αυτό αποτελεί παράδειγμα black-box συμπεριφοράς - ο χρήστης βλέπει ένα αποτέλεσμα αλλά δεν κατανοεί τη λογική πίσω από αυτό. Σύμφωνα με τη διάλεξη 7 (Responsible AI), ένα σύστημα AI πρέπει να παρέχει διαφανή και κατανοητά αποτελέσματα. Για αυτό προστέθηκε ένα **Risk Score Breakdown panel** που αναλύει το τελικό score στα δύο συστατικά του.

Αλλαγές στο app.py:

Η συνάρτηση calculate_risk() τροποποιήθηκε ώστε να επιστρέφει δύο τιμές αντί για μία - το combined risk score και το heuristic score ξεχωριστά:

Πριν - επέστρεφε μόνο ένα αριθμό:

```
def calculate_risk(...) -> float:
    ...
    return round(min(1.0, 0.68 * model_probability + 0.32 * heuristic_score), 4)
```

Μετά - επιστρέφει tuple με και τα δύο:

```
def calculate_risk(...) -> tuple[float, float]:
```

```
...
combined = round(min(1.0, 0.68 * float(model_probability) + 0.32 * heuristic_score),
4)
return combined, round(heuristic_score, 4)
```

Αντίστοιχα ενημερώθηκε η κλήση της συνάρτησης και το context που περνά στο template:

Πριν:

```
risk_score = calculate_risk(model_probability, cues, header_anomalies, feature_map)
```

Μετά:

```
risk_score, heuristic_score = calculate_risk(
    model_probability, cues, header_anomalies, feature_map
)
```

Και στο context.update() προστέθηκε:

```
"heuristic_score": heuristic_score_raw,
```

Αλλαγές στο templates/upload.html:

Προστέθηκε το CSS για τα νέα breakdown boxes και το αντίστοιχο HTML panel που εμφανίζει τις τρεις τιμές ξεχωριστά:

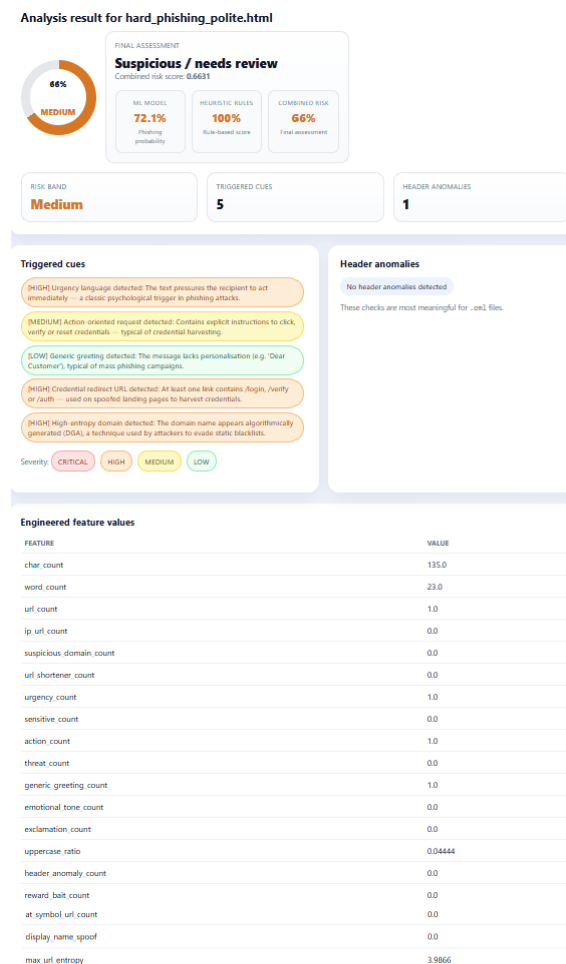
```
<div class="breakdown">
  <div class="breakdown-box">
    <div class="label">ML Model</div>
    <div class="val">{{ (model_probability * 100)|round(1) }}%</div>
    <div class="sub">Phishing probability</div>
  </div>
  <div class="breakdown-box">
    <div class="label">Heuristic Rules</div>
```

```

<div class="val">{{ heuristic_score }}%</div>
<div class="sub">Rule-based score</div>
</div>
<div class="breakdown-box">
  <div class="label">Combined Risk</div>
  <div class="val">{{ risk_percent }}%</div>
  <div class="sub">Final assessment</div>
</div>
</div>

```

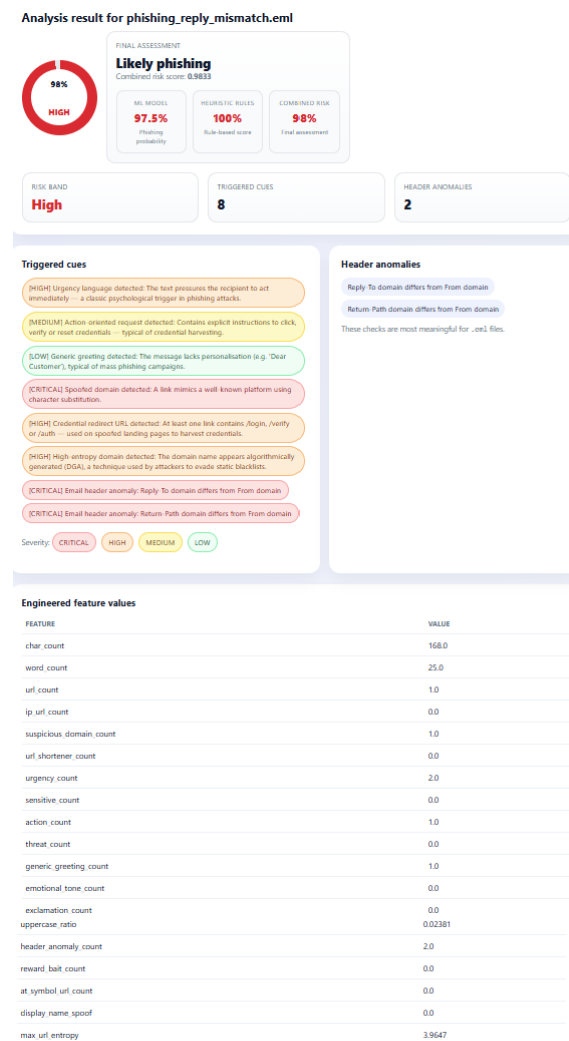
Αποτέλεσμα:



Όπως φαίνεται στο παραπάνω screenshot, ο χρήστης βλέπει πλέον ξεχωριστά την πιθανότητα του ML μοντέλου (72.1%), το heuristic score (32%) και το τελικό combined risk (59%). Αυτό επιτρέπει στον αναλυτή να κατανοεί άμεσα αν η απόφαση

οδηγείται κυρίως από το ML ή από τους κανόνες - βελτιώνοντας την ερμηνευσιμότητα και εκπαιδευτική αξία του συστήματος.

Επιπλέον, τα triggered cues εμφανίζονται πλέον με χρωματική κωδικοποίηση ανά επίπεδο σοβαρότητας: κόκκινο για [CRITICAL], πορτοκαλί για [HIGH], κίτρινο για [MEDIUM] και πράσινο για [LOW]. Ένα legend στο κάτω μέρος του panel επεξηγεί τη χρωματική κλίμακα, επιτρέποντας στον χρήστη να εκτιμά άμεσα τη σοβαρότητα κάθε ένδειξης χωρίς να διαβάσει το πλήρες κείμενο.



Ως προαιρετικό task, χρησιμοποιήθηκε το challenge_dataset.csv -ένα σύνολο adversarial δειγμάτων- για την αξιολόγηση της γενίκευσης του μοντέλου σε άγνωστα δεδομένα εκτός του training set. Τα αποτελέσματα (70% accuracy έναντι 100% στο κανονικό test set) αναδεικνύουν τη διαφορά μεταξύ απόδοσης σε καθαρά datasets και πραγματικών συνθηκών - μια κρίσιμη διάκριση για συστήματα κυβερνοασφάλειας.

Συμπέρασμα

Η παρούσα εργασία επέκτεινε ένα υπάρχον σύστημα ανίχνευσης phishing μέσα από έξι διαδοχικά tasks, καθένα από τα οποία αντιμετώπισε συγκεκριμένη αδυναμία του αρχικού συστήματος.

Στο επίπεδο της εξαγωγής χαρακτηριστικών (Task 1) προστέθηκαν τέσσερα νέα features που καλύπτουν κατηγορίες επιθέσεων αόρατες για το baseline: γλώσσα δειλεασμού, τεχνική παρακαμπτήριου διαπιστευτηρίων μέσω @ σε URLs, πλαστογράφιση display name αποστολέα και ανίχνευση αλγοριθμικά παραγόμενων domains μέσω Shannon Entropy. Οι αντίστοιχοι κανόνες ανίχνευσης (Task 2) εμπλουτίστηκαν με αναλυτικές εξηγήσεις και ετικέτες σοβαρότητας, μειώνοντας το φαινόμενο alert fatigue που αναλύθηκε στη διάλεξη 1.

Η σύγκριση μοντέλων (Tasks 3 και 4) ανέδειξε τον SVM ως καταλληλότερο αλγόριθμο. Επέτυχε 100% accuracy στο κανονικό test set έναντι 99.8% του Logistic Regression, εξαλείφοντας το False Negative της baseline έκδοσης. Η ανάλυση ασφαλιμάτων (Task 5) αποκάλυψε ωστόσο ότι και ο SVM αδυνατεί να ανιχνεύσει adversarial phishing που αποφεύγει σκόπιμα τις γνωστές λέξεις-κλειδιά - επιβεβαιώνοντας την παρατήρηση της Διάλεξης 2 για την ευπάθεια των ML μοντέλων σε adversarial attacks.

Τέλος, η επέκταση της εφαρμογής (Task 6) μετέτρεψε το σύστημα από black-box σε διαφανή πλατφόρμα ανάλυσης, σύμφωνα με την αρχή της Explainability που ορίζεται στη διάλεξη 7 (Responsible AI). Το Risk Score Breakdown panel επιτρέπει στον χρήστη να κατανοεί ποιο από τα δύο συστατικά -ML ή heuristic rules- οδηγεί την κάθε απόφαση. Συνολικά, η εργασία απέδειξε ότι η σύγχρονη κυβερνοάμυνα απαιτεί συνδυασμό στατιστικής πρόβλεψης, ερμηνευτικών κανόνων και διαφάνειας - χωρίς κανένα από τα τρία να είναι από μόνο του επαρκές.